23CSE455 - Design Patterns

Design Pattern Identification and Evolution in a Ruby on Rails-Based Travel Planning Application

Case Study Report - Team B5

Roll number	Student	Batch
AM.SC.U4CSE23301	Aakash Choudhary	CSE D
AM.SC.U4CSE23331	M Tanmayi	CSE D
AM.SC.U4CSE23333	Manish S D A	CSE D
AM.SC.U4CSE23351	S Hemarshitha	CSE D
AM.SC.U4CSE23356	Shreya E R	CSE D

Application: DaPlanStan
Original repository: https://github.com/GeertClaes/daplanstan
Faculty repository: https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy
Final verification date: 11 October 2026

Batch confirmed by the team as CSE D.

Contents

1 Application and Architecture **2**
1.1 Application and source baseline . . . 2
1.2 Architecture and scope . . . 2
1.3 Run and final verification . . . 3

2 Part 1 - Pattern Discovery and Verification **3**
2.1 Method . . . 3
2.2 Frozen baseline totals . . . 3
2.3 Baseline pattern inventory . . . 4
2.4 Evidence records for the nine instances . . . 5
2.5 Per-instance origin, diagram and human verification . . . 8

3 Part 2 - Software Evolution and Refactoring **9**
3.1 CH01 - Personalized itinerary generation . . . 9
3.2 CH02 - Multiple booking providers . . . 10
3.3 CH03 - Different cancellation policies . . . 10
3.4 Updated counts and before/after metrics . . . 11
3.5 Runtime-discovered corrections and diagrams . . . 11

4 Part 3 - Cross-Language Comparison **11**
4.1 Static and dynamic comparison . . . 12

5 LLM Records and Human Verification **12**
5.1 LLM records . . . 12
5.2 Human verification . . . 13

6 Findings, Contributions and Limitations **13**
6.1 Findings . . . 13
6.2 Student contributions . . . 13
6.3 Limitations . . . 14

7 Team Folder and Datasets **14**
7.1 GitHub team-folder screenshot . . . 14

8 References **15**

A Final Dataset Row Counts **15**

B Final Runtime Command **15**

1 Application and Architecture

1.1 Application and source baseline

DaPlanStan is a self-hosted Ruby on Rails travel-planning application for shared trips. The inspected baseline supports trip creation and membership, email-based booking intake, AI-assisted parsing, itinerary items, map/geocoding workflows, expense tracking, OAuth identity linking, notifications, background jobs and mail delivery.

The source repository is `https://github.com/GeertClaes/daplanstan`. The team recorded baseline commit `da54d9ab8bfc118e99375fdd6c506efb20b66e5b`. The source archive was supplied and first inspected on 8 October 2026. The supplied source ZIP has no `.git` directory and does not independently establish that archive-to-commit correspondence; the hash is therefore retained as the stated analysis baseline rather than claimed as archive-verified Git metadata.

Environment. The source requires Ruby 3.4.2 and locks Rails 8.1.3. SQLite is used through Active Record. Other important framework/provider boundaries include Turbo/Stimulus, Action Mailbox, Action Mailer, Active Job/Solid Queue, OmniAuth, Anthropic, Geocoder/Nominatim, Pexels and Active Storage. The source license is the O'Saasy License.

1.2 Architecture and scope

The architecture was traced from routes, controllers, models, jobs, mailboxes, mailers, initializers and database configuration. The major flow is browser/request to Rails controller, domain model persistence and background jobs; inbound mail routes through Action Mailbox and `TripMailbox`; parsed confirmations become `InboxItem` records and may create/link `TripItem` and `Expense`; geocoding delegates to the `Geocodable` concern and Geocoder/Nominatim; OAuth callbacks are coordinated through `SessionsController` and `UserIdentity`.

The source does not justify treating `Trip` as a formally proven DDD aggregate. `TripItem.kind` represents multiple itinerary kinds rather than separate active Flight/Hotel/Activity domain models. `Expense` is expense tracking, not verified payment processing. No baseline cancellation workflow was found.

Faculty investigation area	Source-supported finding at the original baseline
Destination management	Trip destination data, cover-image lookup and geocoding/map workflows were inspected.
Itinerary management	TripItem creation, import, timeline and map flows were inspected.
Hotel/flight/activity booking	Booking-confirmation email intake and parsed itinerary records exist; no active multi-provider booking API integration existed at baseline.
Customer management	User, Traveler, Account and TripMember flows were inspected.
Payment	Expense records support expense tracking; no verified payment-processing workflow was found.
Cancellation	No baseline cancellation-policy or provider-cancellation workflow was found.
Notification	Action Mailer/Mail observer logging and trip-scoped Turbo refresh notifications were inspected.

Table 1: Coverage of the Part 1 investigation scope specified in the case-study brief.

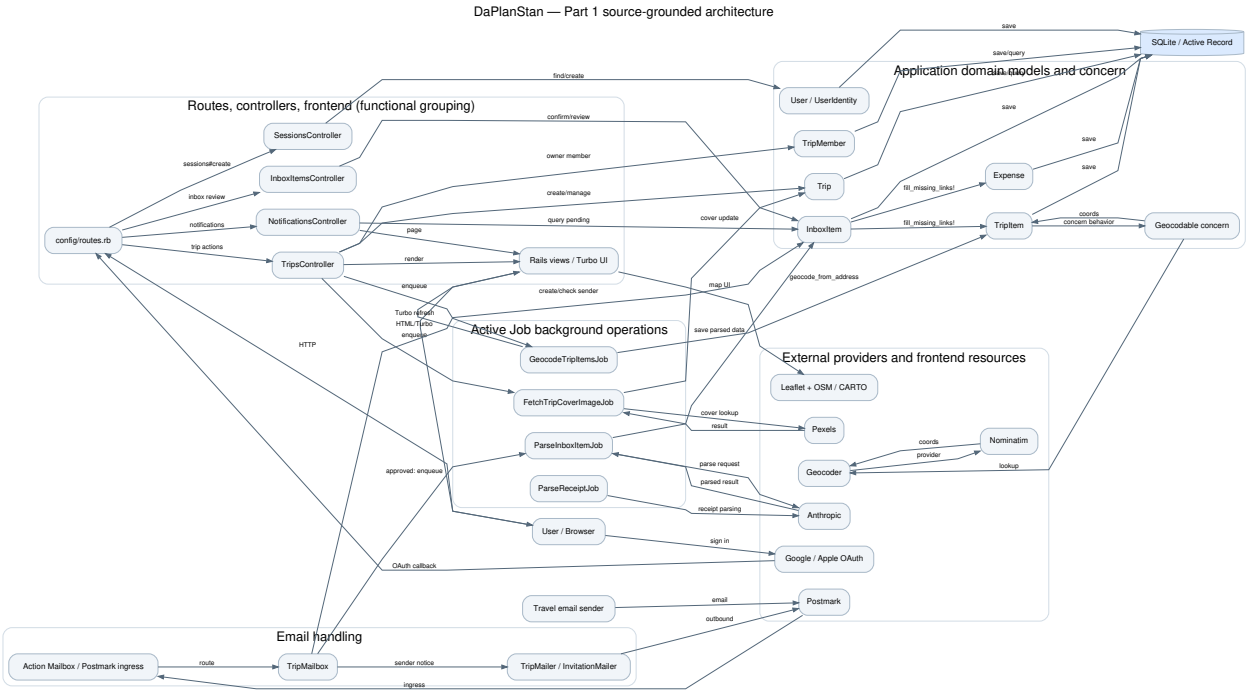


Figure 1: Source-traced DaPlanStan architecture. Editable Mermaid source and SVG are retained under Part1/architecture/.

1.3 Run and final verification

The original repository README describes normal setup with `bundle install`, Rails database creation/migration and `bin/dev`. For final coursework verification, the team installed Ruby 3.4.2 on Windows and resolved native-gem compilation with GNU17 build flags. After environment and test-harness issues were isolated, the cumulative evolved source passed the complete Rails test suite:

59 runs, 359 assertions, 0 failures, 0 errors, 0 skips

Rails 8.1.3 with Puma 7.2.0 also booted successfully in the test environment and listened on port 3000 before a clean shutdown. The raw log is `data/test_result/rails/rails_test_pass.log`. Historical packaging-time logs that state Rails could not execute are retained as historical records and are superseded by this later successful runtime verification.

2 Part 1 - Pattern Discovery and Verification

2.1 Method

Pattern candidates were identified only after tracing responsibilities and collaborations in the source. Each candidate was challenged against pattern intent and participant relationships rather than accepted because of a class name, Rails convention or LLM suggestion. Framework-owned roles were explicitly separated from application-owned decisions. Rejected candidates were retained in supporting evidence so the final inventory was not produced by quota seeking.

2.2 Frozen baseline totals

The frozen baseline contains **9 confirmed pattern instances across 3 distinct pattern types**: six Controller (GRASP), one Information Expert (GRASP), and two Observer (GoF).

Following the report template's explicit counting fields, a separate deduplication audit records **8 concrete participant classes implementing the variable pattern roles**, **76 distinct mapped participant symbols** (classes, framework interfaces, functions/methods and mapped view/runtime symbols), **5 rejected candidates**, and **0 uncertain candidates**. The 76-symbol count is generated from the final P01–P09 participant mappings and is reproducible at `Part1/evidence/participant_symbol_audit.csv`; the 8 concrete variable-role classes are listed at `Part1/evidence/concrete_participant_class_audit.csv`. Rejected candidates and reasons are preserved at `Part1/evidence/rejected_candidates.md`.

Pattern type	Instances	Category
Controller (GRASP)	6	GRASP responsibility assignment
Information Expert (GRASP)	1	GRASP responsibility assignment
Observer (GoF)	2	Behavioral

Table 2: Frozen baseline pattern counts.

Report-template baseline measure	Count
Confirmed pattern types	3
Confirmed pattern instances	9
Concrete participant classes (variable roles; deduplicated)	8
Unique mapped participant symbols	76
Rejected candidates	5
Uncertain candidates	0

Table 3: Additional baseline totals required by the faculty report template.

2.3 Baseline pattern inventory

ID	Pattern	Collaboration	Concrete variable-role class
P01	Information Expert	Parsed Email to Trip Record Linking	InboxItem
P02	Controller	Inbound Travel Email Intake Coordination	TripMailbox
P03	Controller	Trip Creation Coordination	TripsController
P04	Controller	Trip Membership Management Coordination	TripMembersController
P05	Controller	Bulk Itinerary Import Coordination	TripItemsController
P06	Controller	OAuth Sign-in and Identity Linking Coordination	SessionsController
P07	Controller	Account Traveler Enrollment Coordination	TravelersController
P08	Observer	Outbound Mail Delivery Logging	MailLogger
P09	Observer	Trip-Scoped Turbo Refresh Notifications	Framework-mediated subscriber; no additional application concrete class counted

Rejected/uncertain candidates include geocoding as Template Method/Strategy/Adapter, timeline-map Observer, and Beta Invite eligibility as Information Expert. These were rejected because the inspected application interactions did not establish the required pattern collaboration. P08 and P09 explicitly rely in part on framework contracts, which is recorded in their evidence rather than hidden.

2.4 Evidence records for the nine instances

The following records summarize the exact participant and location text preserved in the canonical baseline dataset. Full machine-readable records are in `data/patterns.csv` and editable/rendered UML is in `Part1/architecture/baseline_pattern_uml/`.

P01 - Parsed Email to Trip Record Linking

Pattern/category: Information Expert (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: InboxItem#parsed_data and #parsed_data= = stored extraction interface; InboxItem#confirm_and_create_items! = confirmation and shortlist processing; InboxItem#fill_missing_links! = Information Expert responsible for matching and linking records; InboxItem#build_expense = conditional expense creation; InboxItemsController#update = requesting client; Trip = parent domain context; TripItem = itinerary record; Expense = expense record; TripMember and Traveler = membership and payer lookup; User = reviewing user. Framework: Active Record = associations and persistence.

Source locations: `app/models/inbox_item.rb:19-25,34-58,67-187,191-207;app/controllers/inbox_items_controller.rb:16-35;app/models/trip.rb:4-12;app/models/trip_item.rb:1-28;app/models/expense.rb:1-28;app/models/trip_member.rb:1-6;app/models/traveler.rb:1-8`

Intent/evidence: InboxItem holds parsed travel-email data and is assigned responsibility for interpreting it to match or create TripItem records and conditionally create linked Expense records. InboxItemsController delegates confirmation and relinking to InboxItem.

P02 - Inbound Travel Email Intake Coordination

Pattern/category: Controller (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: ApplicationMailbox = inbound routing declaration; TripMailbox#process = system-event coordinator; TripMailbox#extract_text_body and #extract_html_body = message-content extraction; TripMailbox#notify_owners_of_pending_sender = owner-notification coordination; Trip = recipient-address destination; ApprovedSender = sender-approval record; TripMember and Traveler and User = membership and sender-identity checks; InboxItem = persisted incoming message; ParseInboxItemJob#perform = delegated parsing operation; TripMailer#pending_sender_notification = delegated notification construction. Framework: Action Mailbox = message routing and processing lifecycle; Active Record = persistence; Active Job = background execution; Action Mailer = notification delivery infrastructure. Provider: Anthropic = subsequent AI extraction through ParseInboxItemJob.

Source locations: `app/mailboxes/application_mailbox.rb:1-3;app/mailboxes/trip_mailbox.rb:2-31,35-60;app/models/trip.rb:4-19;app/models/approved_sender.rb:1-6;app/models/inbox_item.rb:1-17;app/models/trip_member.rb:1-6;app/jobs/parse_inbox_item_job.rb:99-112,127-139;app/mailers/trip_mailer.rb:2-10`

Intent/evidence: TripMailbox#process coordinates the incoming-email event by identifying the destination Trip through its inbound address and checking sender eligibility. It creates an InboxItem and delegates approved messages to parsing or unapproved messages to owner notification.

P03 - Trip Creation Coordination

Pattern/category: Controller (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: TripsController#create = system-event coordinator; ApplicationController#current_user and #current_account = authenticated context; Trip = new domain

object with validation and inbound-address generation; Trip#generate_inbound_email and #build_inbound_email = address-generation operations; Account#owner_traveler = owner-traveler lookup; Traveler = selected account participant; TripMember = initial owner-role membership; FetchTripCoverImageJob#perform = delegated cover-image retrieval. Framework: Rails routing and Active Record = request handling and persistence; Active Job = job execution. Provider: Pexels = external image search.

Source locations: `app/controllers/trips_controller.rb:35-48;app/controllers/application_controller.rb:14-20;app/models/trip.rb:4-19,31-37;app/models/account.rb:9-11;app/models/traveler.rb:1-8;app/models/trip_member.rb:1-6;app/jobs/fetch_trip_cover_image_job.rb:4-32;config/routes.rb:21`

Intent/evidence: TripsController#create coordinates the trip-creation event by saving the new Trip and creating its initial owner-role TripMember through Account#owner_traveler. It then delegates cover-image retrieval to FetchTripCoverImageJob.

P04 - Trip Membership Management Coordination

Pattern/category: Controller (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: TripMembersController#create and #destroy = membership-event handlers; TripMembersController#set_trip = trip resolution; TripMembersController#require_owner_or_planner = authorization; TripMembersController#add_traveler = duplicate checking and membership creation; TripMembersController#resolve_and_add_by_email = traveler resolution and invitation coordination; ApplicationController#current_member_for = acting-user membership lookup; Trip and Account = trip and account context; Traveler and User = participant identity; TripMember = role-bearing membership; InvitationMailer#invite_to_trip = invitation construction. Framework: Rails routing and Active Record = request handling and association persistence; Action Mailer and Active Job = mail infrastructure.

Source locations: `app/controllers/trip_members_controller.rb:6-22,26-65;app/controllers/application_controller.rb:22-25;app/models/trip.rb:4-12;app/models/account.rb:1-11;app/models/traveler.rb:1-8;app/models/user.rb:11-17;app/models/trip_member.rb:1-6;app/mailers/invitation_mailer.rb:8-13;config/routes.rb:40`

Intent/evidence: TripMembersController coordinates addition and removal of members in an existing Trip. It checks owner or planner authorization and resolves account travelers before creating contributor-role memberships or removing permitted memberships. Invitation delivery is delegated to InvitationMailer.

P05 - Bulk Itinerary Import Coordination

Pattern/category: Controller (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: TripItemsController#import = import-event coordinator for preview and confirmation; TripItemsController#bulk_create_items = batch matching and persistence coordination; TripItemsController#parse_import_time = imported date-time parsing; Trip = destination context; TripItem = existing or newly imported itinerary record; Geocodable#geocoded? = coordinate-completeness check; GeocodeTripItemsJob#perform = delegated geocoding; import.html.erb = preview and confirmation interface. Framework: Rails routing and Active Record = queries and persistence; Active Job = deferred execution.

Source locations: `app/controllers/trip_items_controller.rb:31-58,128-179;app/models/trip.rb:4-12;app/models/trip_item.rb:1-28;app/models/concerns/geocodable.rb:9-11;app/jobs/geocode_trip_items_job.rb:4-21;app/views/trip_items/import.html.erb:93-106;config/routes.rb:30-34`

Intent/evidence: TripItemsController#import coordinates JSON preview and confirmed batch

import as one use case. Its bulk_create_items operation checks names and duplicates and creates or updates itinerary ideas before scheduling GeocodeTripItemsJob for missing coordinates.

P06 - OAuth Sign-in and Identity Linking Coordination

Pattern/category: Controller (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: SessionsController#create = post-authentication system-event coordinator; SessionsController#provision_account_or_gate = account-eligibility coordination; SessionsController#create_account_for = account and initial-traveler creation; UserIdentity = external provider identity record; User = local authenticated identity; Traveler#accept! = pending invitation association; Account = user-owned account; Invite#usable? and #email_matches? = invite-eligibility predicates; ApplicationController#current_user = session identity lookup; OmniAuth initializer = provider configuration. Framework: OmniAuth and provider integrations = OAuth processing and normalized authentication data; Rails sessions and Active Record = local session and persistence.

Source locations: `app/controllers/sessions_controller.rb:4-41,54-81;app/models/user_identity.rb:1-8;app/models/user.rb:11-22;app/models/traveler.rb:26-28;app/models/account.rb:1-11;app/models/invite.rb:12-18;app/controllers/application_controller.rb:14-16;config/initializers/omniauth.rb:1-16;config/routes.rb:12-14`

Intent/evidence: SessionsController#create coordinates post-OAuth identity handling by finding a UserIdentity through provider and UID or resolving a User by email. It creates missing identity links and establishes the session before accepting pending travelers and applying account-provisioning or invitation rules.

P07 - Account Traveler Enrollment Coordination

Pattern/category: Controller (GRASP); GRASP - Responsibility Assignment.

Participant mapping: Application: TravelersController#create = account-enrollment system-event coordinator; TravelersController#set_account = account-owner access control; Account = account-scoped traveler context; Traveler = new account participant; User = registered identity lookup; InvitationMailer#invite = account invitation construction; settings/show.html.erb = enrollment interface. Framework: Rails routing and Active Record = request handling and persistence; Action Mailer and Active Job = invitation delivery infrastructure.

Source locations: `app/controllers/travelers_controller.rb:5-35,54-56;app/models/account.rb:1-11;app/models/traveler.rb:1-8,18-28;app/models/user.rb:11-22;app/mailers/invitation_mailer.rb:2-6;app/views/settings/show.html.erb:114-127;config/routes.rb:48-50`

Intent/evidence: TravelersController#create coordinates account-level traveler enrollment by validating supplied identity information and checking existing participants. It creates name-only or email-based Traveler records and links registered Users or requests invitations when applicable. This does not itself establish TripMember membership.

P08 - Outbound Mail Delivery Logging

Pattern/category: Observer (GoF); GoF - Behavioral (framework-mediated).

Participant mapping: Application: MailLogger = concrete observer; MailLogger.delivered_email(message) = notification callback; ActionMailer::Base.register_observer(MailLogger) = application registration call; InvitationMailer#invite and #invite_to_trip = mail-producing collaborators; TripMailer#pending_sender_notification = mail-producing collaborator. Framework: ActionMailer::Base.register_observer = observer registration interface; Mail gem = publisher and observer

registry and notification dispatch; Action Mailer = mail lifecycle. Provider: Postmark = external delivery transport and not an Observer participant.

Source locations: `config/initializers/mail_logger.rb:1-7;app/mailers/invitation_mailer.rb:2-13;app/mailers/trip_mailer.rb:2-10;app/mailers/application_mailer.rb:1-5;Gemfile.lock:19,185`

Intent/evidence: DaPlanStan defines MailLogger.delivered_email and explicitly registers MailLogger through ActionMailer::Base.register_observer. The Mail framework invokes the registered concrete observer during the delivery lifecycle so individual mailers do not directly call the logger.

P09 - Trip-Scoped Turbo Refresh Notifications

Pattern/category: Observer (GoF); GoF - Behavioral (framework-mediated).

Participant mapping: Application: InboxItem = model publication source through broadcasts_refreshes_to :trip; GeocodeTripItemsJob#perform = explicit broadcast source through Turbo::StreamsChannel.broadcast_refresh_to(trip); trips/show.html.erb = trip-page subscription and refresh configuration; inbox_items/index.html.erb = inbox-page subscription. Framework: Turbo::Broadcastable = model-triggered broadcast implementation; Turbo::StreamsChannel = trip-stream subscription and publication channel; Turbo::Streams::Broadcasts = refresh broadcast implementation; Action Cable = stream delivery infrastructure; Turbo browser runtime = concrete stream subscriber and refresh reaction.

Source locations: `app/models/inbox_item.rb:1-5;app/jobs/geocode_trip_items_job.rb:4-21;app/views/trips/show.html.erb:2-9;app/views/inbox_items/index.html.erb:1;Gemfile.lock:424;TurboRails2.0.23:app/models/concerns/turbo/broadcastable.rb;TurboRails2.0.23:app/channels/turbo/streams_channel.rb;TurboRails2.0.23:app/channels/turbo/streams/broadcasts.rb`

Intent/evidence: InboxItem changes and GeocodeTripItemsJob completion request refresh broadcasts for a Trip. The trip show and inbox index pages subscribe through turbo_stream_from @trip. Turbo Rails and Action Cable implement subscription and notification delivery as one shared trip-scoped Observer collaboration.

2.5 Per-instance origin, diagram and human verification

Each P01–P09 record above is source-backed. The table below supplies the report-template origin/verification fields without changing the frozen classifications. Human-verification labels are copied from the consolidated team verification sheet; reviewer display aliases are mapped to official team names in `data/test_result/human_verification/REVIEWER_NAME_MAP.md`.

ID	Origin	Human verification	Diagram
P01	Application responsibility; Active Record persistence framework	S Hemarshitha - verified	`Part1/pattern_architecture/baseline_pattern_uml/svg/P01_Parsed_Email_to_Trip_Record_Linking.svg`
P02	Application mailbox coordinator with Rails/mail/job framework support	Manish S D A - verified	`Part1/pattern_architecture/baseline_pattern_uml/svg/P02_Inbound_Travel_Email_Intake_Coordination.svg`

ID	Origin	Human verification	Diagram
P03	Application Rails controller with persistence/job/provider collaborators	Manish S D A - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P03_Trip_Creation_Coordination.svg
P04	Application Rails controller with membership/mailer collaborators	Manish S D A - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P04_Trip_Membership_Management_Coordination.svg
P05	Application Rails controller with persistence/geocoding collaborators	Aakash Choudhary - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P05_Bulk_Itinerary_Import_Coordination.svg
P06	Application Rails controller with OmniAuth framework/provider boundary	S Hemarshitha - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P06_OAuth_Sign in_and_Identity_Linking_Coordination.svg
P07	Application Rails controller with mail delivery infrastructure	M Tanmayi - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P07_Account_Traveler_Enrollment_Coordination.svg
P08	Application concrete observer registered through Action Mailer/Mail	Shreya E R - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P08_Outbound_Mail_Delivery_Logging.svg
P09	Application broadcast/subscription declarations with Turbo/Action Cable framework delivery	Manish S D A - verified	Part1/pattern_architecture/baseline_pattern_uml/svg/P09_Trip_Scoped_Turbo_Refresh_Notifications.svg

3 Part 2 - Software Evolution and Refactoring

Three cumulative realistic changes from the assigned case study were implemented in order. The original baseline remains frozen; each AFTER snapshot is preserved either directly or as an exact input ZIP plus change patch. Evolution status is recorded in `data/patterns_evolution.csv`.

3.1 CH01 - Personalized itinerary generation

Requirement and acceptance criteria. An authenticated trip owner can enter supported interests, budget, pace and notes; receive at most six validated suggestions; preview without persistence; reject with no write; or accept a signed, trip/user-bound preview that is revalidated and persisted only as itinerary `idea` items. Generated payloads cannot inject booking/payment/status fields. Existing import behavior and geocoding remain functional.

Before/after and source. BEFORE is `data/code/original/daplanstan-main/`. The exact Scenario 1 AFTER source used for the next change is preserved as `Part2/code/snapshots/scenari`

`o1_personalized_itinerary_AFTER.zip`. The main source diff is `Part2/changes/Change1/source.patch` and the full change record is `Part2/changes/Change1/SCENARIO1_CHANGE_RECORD.md`.

Implementation and tests. `ItinerarySuggestionGenerator` validates preferences and suggestions, calls the Anthropic tool contract, and returns a transient proposal. `TripItemsController` coordinates preview/confirmation and reuses batch persistence. Historical isolated checks recorded **6 runs / 20 assertions**; the Rails environment was then unavailable, so no historical Rails pass is invented. The final cumulative source was later included in the all-green Rails verification.

Pattern effects and refactoring. P05 Bulk Itinerary Import Coordination is **modified**; S1-N01 Personalized Itinerary Generation Coordination is **newly introduced** as Controller (GRASP). The refactoring replaces global callback manipulation with instance-scoped deferred geocoding. Snapshot total: **10 instances / 3 types**.

3.2 CH02 - Multiple booking providers

Requirement and acceptance criteria. Two differently shaped provider inputs must be selectable by an authenticated trip owner and normalized to one application-facing offer contract. Provider-specific fields and sample amounts are re-resolved from server-owned local data rather than trusted from the browser. Adding a sample offer creates only an itinerary `idea`; it must not perform a real reservation, payment or confirmation.

Before/after and source. BEFORE is the Scenario 1 ZIP above. The exact Scenario 2 AFTER input for CH03 is preserved at `Part2/code/snapshots/scenario2_multiple_booking_providers_AFTER.zip`. Diff and source verification are `Part2/changes/Change2/source.patch` and `Part2/changes/Change2/Scenario2_AFTER_source_verification.md`.

Implementation and tests. `BookingProviders::AtlasAdapter` and `BeaconAdapter` normalize distinct fixture schemas through the Registry; `BookingOptionsController` coordinates option display and sample-item creation. Isolated provider checks recorded **10 runs / 67 assertions**. Final Rails execution later found one duplicate-marker lookup defect, which was corrected to exact substring matching before the final all-green run.

Pattern effects. S2-N01 Multiple Booking Provider Normalization is one **Adapter (GoF)** collaboration containing both concrete adapters; S2-N02 Multiple Booking Provider Option Coordination is one new **Controller (GRASP)**. Snapshot total: **12 instances / 4 types**. Two provider adapters are not counted as two Adapter pattern instances.

3.3 CH03 - Different cancellation policies

Requirement and acceptance criteria. Eligible confirmed email-linked TripItems can receive deterministic hypothetical cancellation quotes under Flexible, Moderate or Strict policies. Exact 72-hour and 24-hour boundaries are fixed; unknown policies and ineligible/tampered data are rejected; server-loaded amount/start/currency are used; money uses BigDecimal half-up rounding; GET/POST are quote-only and perform no cancellation, refund, booking-state write or external provider call.

Before/after and source. BEFORE is the Scenario 2 ZIP. The final corrected cumulative source is stored directly at `Part2/code/final/`. Diff/source verification are `Part2/changes/Change3/source.patch` and `Part2/changes/Change3/Scenario3_AFTER_source_verification.md`.

Implementation and tests. `CancellationPolicies::QuoteEvaluator` selects a whitelisted policy and computes fee/refund; the three policies implement one fee-rate protocol; `CancellationQuotesController` coordinates owner-authorized quotes. Isolated checks recorded **10 runs / 45 assertions**. The final cumulative Rails suite then passed **59 runs / 359 assertions / 0 failures / 0 errors / 0 skips**.

Pattern effects. S3-N01 Cancellation Policy Evaluation is one **Strategy (GoF)** collaboration containing three concrete strategies; S3-N02 Cancellation Quote Coordination is one new **Controller**

(GRASP). Final total: **14 instances / 5 types**.

3.4 Updated counts and before/after metrics

The same LOC rule is applied to each snapshot: include `app/**/*.rb` and `config/routes.rb`; exclude blank/comment-only lines, tests, views, generated database files, documentation and evidence scripts. The reproducible method is `Part2/evidence/EVOLUTION_METRICS_METHOD.md`.

Snapshot	Source LOC	Instances	Types	Concrete variable-role classes	Actual execution evidence
Baseline	1797	9	3	8	Supplied baseline had no application test suite
CH01	2011	10	3	8	6 runs / 20 assertions isolated; historical Rails blocked
CH02	2197	12	4	11	10 runs / 67 assertions isolated; historical Rails blocked
CH03 final	2332	14	5	15	10 runs / 45 assertions isolated; final cumulative Rails 59/359, 0 failures/errors

Table 6: Evolution metrics using one fixed measurement scope.

The increase in LOC is reported as descriptive evidence only. It does not by itself prove better design. Benefits instead come from explicit separation of new variability: generated-itinerary coordination, provider normalization and interchangeable cancellation rules. Trade-offs include more application code, fictional provider/cancellation assumptions, and no live provider/payment/cancellation integration.

3.5 Runtime-discovered corrections and diagrams

Final Rails runtime verification exposed one Scenario 2 duplicate-detection query defect and two Scenario 1 Minitest stub-compatibility issues. These corrections are documented at `data/test_result/rails/RUNTIME_ISSUE_RESOLUTION.md`. The final source at `Part2/code/final/` contains the corrected query and compatible test stubs. Editable DOT and rendered SVG diagrams for S1-N01, S2-N01, S2-N02, S3-N01 and S3-N02 are in `Part2/diagrams/`.

4 Part 3 - Cross-Language Comparison

The short case-study brief requested two significant patterns. The later detailed faculty CSV instruction required one representative confirmed instance for each detected pattern type. The team followed the stricter requirement: one representative for all five confirmed types, each implemented in Java, Python, JavaScript and C++ using the same language-neutral fixtures and expected outputs.

Pattern	Representative source collaboration
Controller	Trip Creation Coordination (P03)
Information Expert	Parsed Email to Trip Record Linking (P01)
Observer	Outbound Mail Delivery Logging (P08)
Adapter	Multiple Booking Provider Normalization (S2-N01)
Strategy	Cancellation Policy Evaluation (S3-N01)

Table 7: Representative source-grounded pattern problems.

The result is **20 implementations**: 5 pattern types multiplied by 4 languages. Recorded shared-fixture results are Strategy 60/60, Observer 28/28, Controller 32/32, Information Expert 32/32 and Adapter 40/40, for **192/192 passing cases**. The exact 20-row faculty dataset is `data/cross_language.csv`.

4.1 Static and dynamic comparison

	Source LOC				Maximum function CC			
Pattern	Java	Python	JS	C++	Java	Python	JS	C++
Strategy	52	54	42	74	12	8	8	15
Observer	47	32	43	60	5	5	5	4
Controller	66	59	55	90	9	10	9	13
Information Expert	63	45	38	83	7	7	6	7
Adapter	120	93	67	141	12	7	8	12

Pattern	Java ms	Python ms	JS ms	C++ ms
Strategy	91.907	839.469	30.426	2.582
Observer	109.548	1133.245	43.673	2.325
Controller	159.473	818.330	37.862	2.566
Information Expert	75.602	819.622	36.436	4.391
Adapter	71.668	852.240	35.246	2.706

Table 9: Recorded median fresh-process times from `data/cross_language.csv`; these include process/test-harness overhead.

The implementations use OpenJDK/javac 21.0.12.1, Python 3.13.5, Node.js 22.16.0 and g++ 14.2.0 with C++17. Timing was measured after compilation where applicable, with one warm-up and five measured fresh-process executions on the same machine. Reported medians include process startup/runtime/test-harness overhead, so they are not pure algorithm benchmarks.

Across the fixed examples, Java and C++ use more explicit types/interfaces in several patterns, while Python and JavaScript express equivalent roles more compactly. These LOC and timing observations are descriptive only and do not establish that one language or pattern implementation is universally superior.

5 LLM Records and Human Verification

5.1 LLM records

The canonical `data/llm.csv` contains **29 provenance-audited interaction records** using exactly the seven required columns: part, model/date, prompt file, input file, output file, evaluation and verification file. The official part names in the final CSV are Part 1 Pattern Discovery, Part 2 Software Evolution and Part 3 Cross-Language Analysis.

The team also supplied authentic full exported ChatGPT conversation PDFs. Per the team's faculty instruction, the PDFs are retained rather than replaced by invented text transcripts:

- `Part1/Part1_prompt.pdf` - official Part 1 pattern-discovery conversation.
- `Part1/llm/Architecture_Chat_ShreyaER.pdf` - application/architecture conversation.
- `Part2/Part2_prompt.pdf` - official Part 2 software-evolution conversation.
- `Part3/llm/part3_prompt.pdf` - official Part 3 cross-language conversation.

- `Part3/llm/Final_Documentation_Chat_MTanmayi.pdf` - final LLM/human-verification/documentation workflow.

Historical exact model configurations were not recoverable for the older interactions, so `data/llm.csv` says `cannot confirm` instead of inventing model names. Interaction-level recovered prompt/output material remains under each part's `llm/audit/` directory. Historical evaluation wording is preserved as audit-time evidence; later human review is recorded separately.

5.2 Human verification

The consolidated human-verification sheet contains **148 items**: **143 verified**, **3 partially verified** and **2 cannot confirm**, dated 2026-10-11. It is preserved at `data/test_result/human_verification/human_verification_final_team.csv` and as the original XLSX companion file. Review assignments were distributed across all five team members; the raw reviewer display names are mapped to official submission names in `data/test_result/human_verification/REVIEWER_NAME_MAP.md`.

One cannot-confirm item concerned application setup/boot at the time of human review. Later machine evidence established successful Rails/Puma boot and a full all-green test suite; the historical human sheet is not retroactively rewritten. The other persistent limitation is the inability to prove that the supplied source ZIP is byte-for-byte the stated Git commit because the archive contains no `.git` metadata.

6 Findings, Contributions and Limitations

6.1 Findings

The case study shows that source-level pattern classification is more reliable when collaborations are counted by responsibility and interaction rather than by class count. The baseline is dominated by GRASP Controllers because Rails controllers coordinate distinct system events, while the two confirmed Observer cases depend partly on framework notification contracts. Evolution adds Adapter and Strategy only when new variability genuinely appears: incompatible provider schemas in Scenario 2 and interchangeable cancellation policies in Scenario 3.

The cumulative inventory grows from 9 instances/3 types to 14 instances/5 types without retroactively relabeling the baseline. Cross-language implementations preserve the same pattern problems and participant responsibilities across four languages, while syntax, type systems, abstraction mechanisms and process overhead differ.

6.2 Student contributions

The team confirmed the following primary workstream ownership. All members also participated in the consolidated human verification/final review recorded in the verification sheet.

Roll number	Student	Recorded contribution
AM.SC.U4CSE23301	Aakash Choudhary	Official Part 2 - Software Evolution and Refactoring
AM.SC.U4CSE23331	M Tanmayi	LLM records, Human Verification and Final Documentation
AM.SC.U4CSE23333	Manish S D A	Official Part 3 - Cross-Language Analysis
AM.SC.U4CSE23351	S Hemarshitha	Official Part 1 - Pattern Discovery and Verification

Roll number	Student	Recorded contribution
AM.SC.U4CSE23356	Shreya E R	Application Understanding and Framework Architecture

6.3 Limitations

- The baseline archive lacks Git metadata; commit correspondence is qualified.
- No live Anthropic API generation was performed in final manual verification. Scenario 1 is covered by controlled tests, and no API key is included.
- Scenario 2 uses fictional Atlas/Beacon fixtures and does not perform a real reservation or payment.
- Scenario 3 is a quote-only demonstration; it does not cancel bookings or issue refunds.
- Optional VIPS loader warnings occurred on Windows but did not block Rails tests or server boot.
- Historical LLM model identifiers are unconfirmed and copied/printed outputs are not represented as byte-exact native exports.

7 Team Folder and Datasets

The final team folder follows the faculty's three official technical parts. Canonical datasets are stored once under `data/`; supporting code/evidence is organized under Part1, Part2 and Part3. No top-level Part4 or Part5 submission stage is used.

- `data/patterns.csv` - 9 frozen baseline collaborations.
- `data/pattern_counts.csv` - per-pattern and total counts across four snapshots.
- `data/patterns_evolution.csv` - 45 versioned collaboration rows.
- `data/cross_language.csv` - 20 language/pattern rows.
- `data/llm.csv` - 29 provenance-audited LLM records.
- `data/code/original/` - original application source.
- `data/test_result/` - Rails runtime and human-verification evidence.
- `Part1/pattern_architecture/`, `Part1/evidence/`, `Part1/llm/` - pattern-discovery support.
- `Part2/code/`, `Part2/changes/`, `Part2/diagrams/`, `Part2/evidence/`, `Part2/llm/` - evolution support.
- `Part3/` - five representative pattern problems, four languages each, common fixtures, execution evidence and LLM material.
- `Part3/llm/B5_slide.pdf` - requested presentation PDF.

7.1 GitHub team-folder screenshot

The following screenshot shows the actual B5 team folder after it was pushed to the team's GitHub fork. The repository view visibly contains the required `Part1/`, `Part2/`, `Part3/`, `data/`, `B5_Report.pdf`, `B5_Report.tex`, and `README.md` entries.

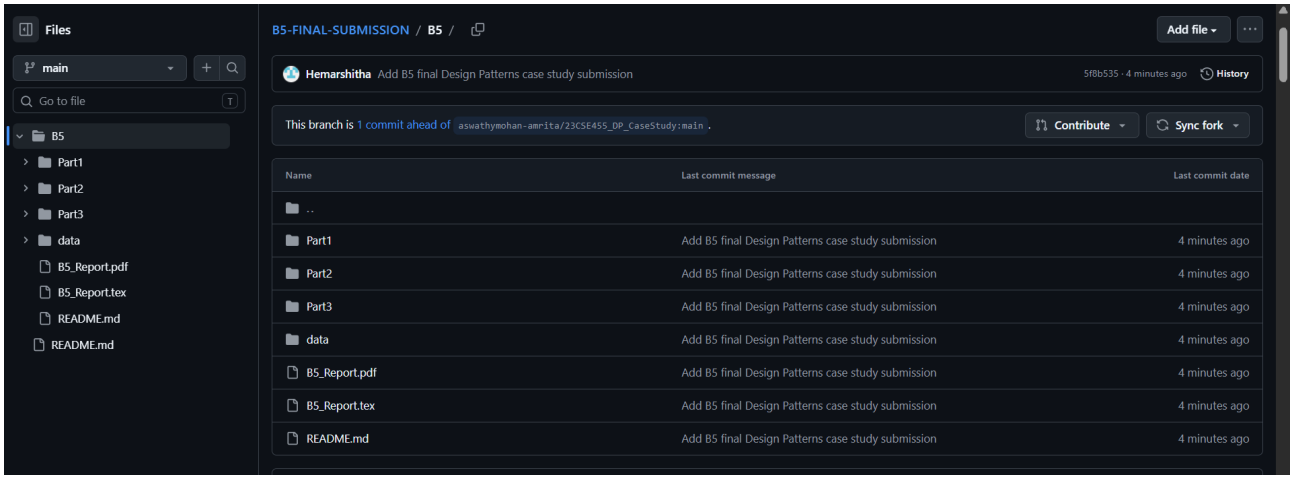


Figure 2: B5 team folder in the GitHub submission repository.

8 References

References

[1] Geert Claes, *DaPlanStan*, source repository, https://github.com/GeertClaes/daplanstan.

[2] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[3] C. Larman, *Applying UML and Patterns*, Prentice Hall. GRASP responsibility-assignment patterns.

[4] Ruby on Rails Guides, https://guides.rubyonrails.org/.

[5] Ruby language documentation, https://www.ruby-lang.org/.

A Final Dataset Row Counts

Dataset	Data rows
patterns.csv	9
pattern_counts.csv	23
patterns_evolution.csv	45
cross_language.csv	20
llm.csv	29

B Final Runtime Command

The successful full-suite verification command on the team's Windows environment was:

```
set "RAILS_ENV=test" && bundle exec ruby \
  -I"C:/Ruby34-x64/lib/ruby/gems/3.4.0/gems/minitest-mock-5.27.0/lib" \
  -e "require './config/environment'; ActiveJob::Base.queue_adapter = :test; \
  load 'bin/rails'" test
```